Artificial Neural Network

Computer science and information technology department
By : Dr Hussein Omron

Lecture 7

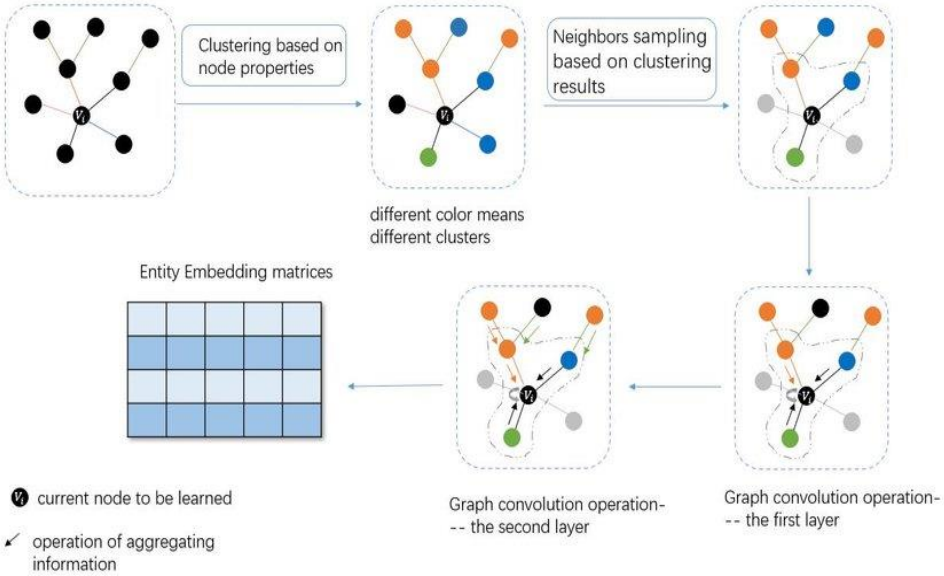
- **GNN Geometric Neural Network**
- **What GMINN**

Lecture includes

1. **Review for GNN**
2. **How it works**
3. **GMINN neural network Problem solution**

GNN Geometric Neural Network

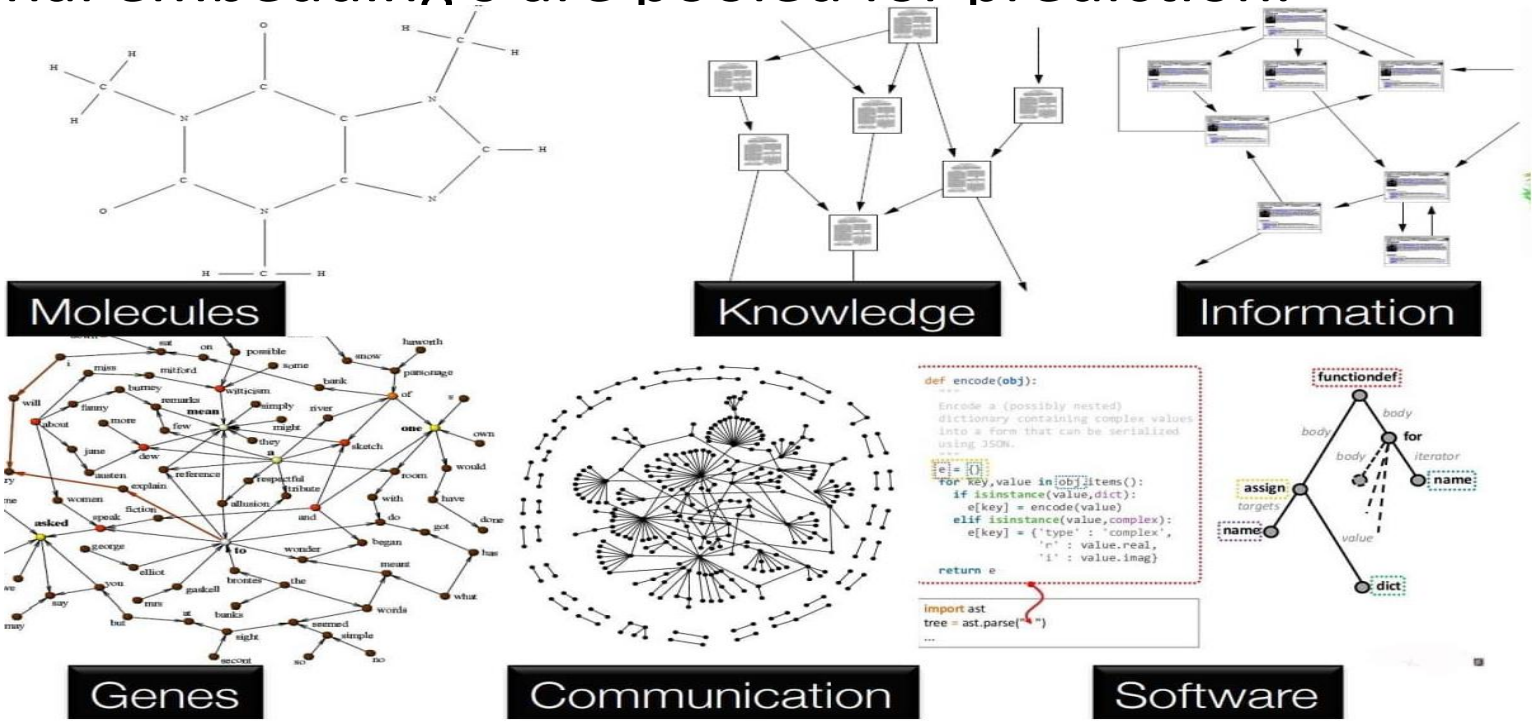
- At years 2000, Euclidean geometry study of invariants or symmetries where Graphs model relations and interactions, such as social networks Nodes represent users, and edges capture relationships
- learning methods are designed to work on data that is represented in a Euclidean space, where points can be easily compared to each other using distance metrics.



Traditional neural networks fail on graph data, GNNs solves by respecting graph structure, as social network, chemistry

Example Workflow

1. Input: Graph with nodes and edges.
2. Each node starts with an initial feature vector.
3. Nodes exchange information with neighbors, message passing
4. Node embedding's are updated layer by layer.
5. Final embedding's are pooled for prediction.



How GNNs Work: Step-by-Step Process of

1. Graph Representation: it consists of **nodes** and **edges**. Each node have features, and edges have relationship strength.

2. Message Passing Aggregation: Each node collects information from its neighbors by **aggregation function** over the features .

3. Update Function: node updates its representation by combines previous state with aggregated neighbor .

4. Layer Stacking: Multiple GNN layers gather **multi-hop** :

1 layer → captures immediate neighbors.

2 layers → captures neighbors of neighbors.

More layers → capture broader graph context.

5. Readout / Pooling: methods include **global mean pooling, sum pooling, or attention pooling**.

6. Training: NNs are trained using **back propagation** with loss functions:

Node classification → predict labels for individual nodes.

Link prediction → predict if an edge exists between two nodes.

Graph classification → predict properties of the entire graph.

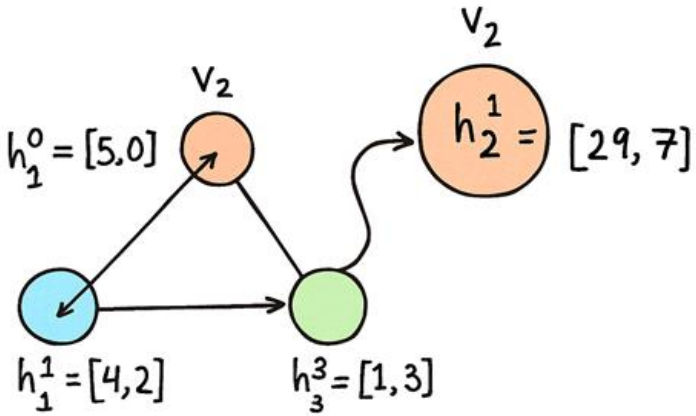
Data as Graphs

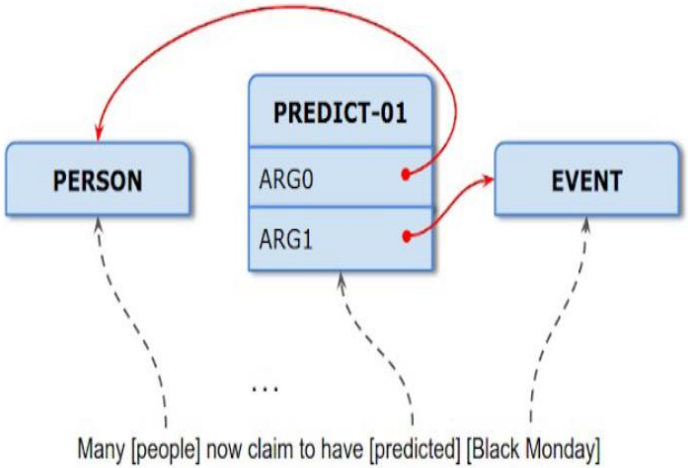
tasks Decomposition for GNNs

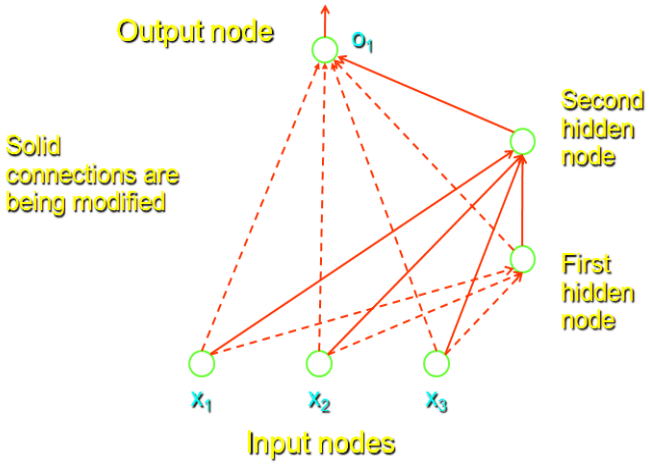
1.Node classification for each node (label , value) node vector= $\mathbf{X}_v \in \mathbb{R}M$

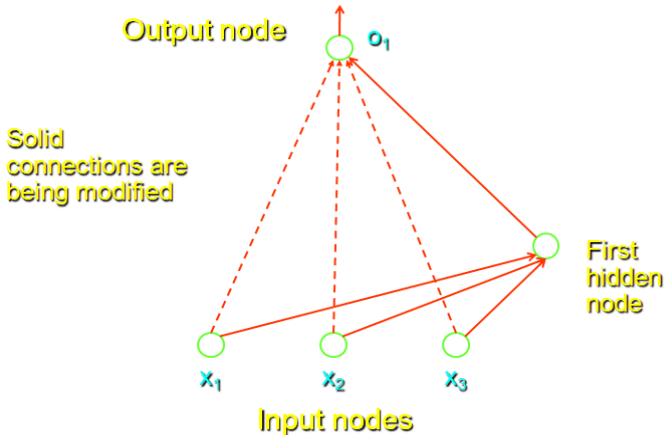
2.Edge classification for each edge (label, link present or absence)

3.Graph classification for entire graph (label ,value)

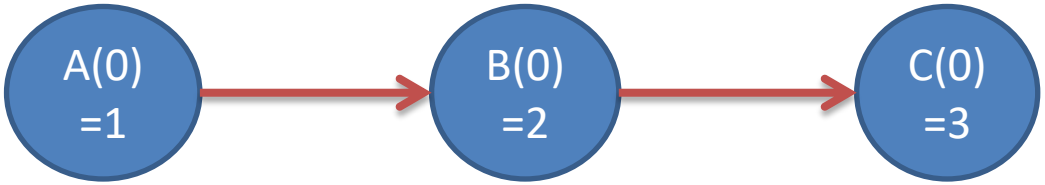








example -node graph:



Nodes: A, B, C

Edges: A–B, B–C (A connected to B, and B connected to C)

Initial Node Features

Let's each node has **1-dimensional feature** (just a number):

$h_A(0)= 1$, $h_B(0)= 2$, $h_C(0)= 3$

Step 1: Message Passing (Aggregation)

Each node collects **average of its neighbors' features**.

Node A's neighbors: {B} → aggregate = $h_B(0)= \mathbf{2}$

Node B neighbor: {A,C} → aggregate $= \frac{[hA(0)+hC(0)]}{2} = \mathbf{2}$

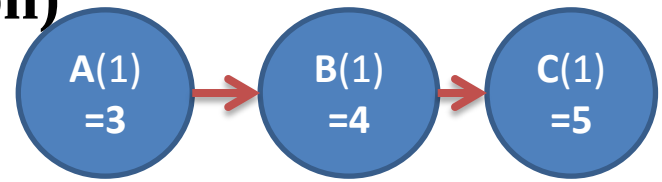
Node C's neighbors: {B} → aggregate = $h_B(0)= 2$

Step 2: Update Function

update each node by combining its **own feature** and **neighbor aggregate**. simple rule: $h_i(1)=h_i(0)+\text{aggregate}$

Node A: $h_A(1)=1+2 = 3$, B: $h_B(1)=2+2 = 4$. C: $h_C(1)=3+2 = 5$

Step 3: Second Layer (Multi-hop Information)



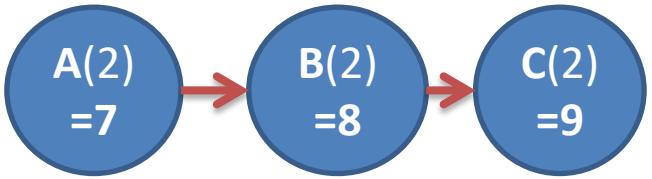
neighbors of neighbors. Repeat aggregation:

Node A's neighbors:{B}→aggregate=$h_B(1)$=4

Node B's neighbors:{A, C}→aggregate = $[h_A(1)+h_C(1)]/2=[3+5]/2=4$

Node C's neighbors: {B} → aggregate = $h_B(1)$= 4

Update again:



Node A: $h_A(2)=3+4= 3 + 4 = 7$

Node B: $h_B(2)= 4 + 4 = 8$

Node C: $h_C(2)=5+4= 5 + 4 = 9$

Final Node Embeddings

After 2 layers of message passing:

$h_A(2)= 7$, $h_B(2)= 8$, $h_C(2)= 9$

Key Takeaway

Each node's representation evolves by **mixing its own features with its neighbors' features**.

After multiple layers, nodes capture **multi-hop structural information**.

This is the essence of how GNNs learn from graph data.

How to Apply GNNs to a 6×6 Image Grid

1. Convert Image to Graph: Each pixel is a **node** graph. 6×6 =**36 node**

Node features: Each pixel intensity RGB value becomes feature vector

Edges: Connect each pixel to neighbors (4-neighbor or 8-neighbor)

Example: Node (3,3) connects to: (3,2), (3,4), (2,3), (4,3) → 4-neighbor

Optionally also diagonals → 8-neighbor

45	23	12	34	9
27	22	61	62	10
25	52	8	14	32
13	33	56	15	25
50	26	16	27	2
2	51	54	21	59

2. Message Passing m_{ij}

Each pixel (node) aggregates features from its neighbors.

Use **mean, sum, or weighted aggregation** similar to convolution, but instead of fixed kernel, the graph structure defines the neighborhood.

3. Update Node Features

Apply neural network (e.g., MLP or linear layer) to combine pixel's own feature with aggregated neighbor features.

$$h_i^{(1)} = \mathrm{MLP}(h_i^{(0)}, \mathrm{Aggregate}(h_j^{(0)} \text{ for } j \in N(i)))$$

4. Stack Layers

GNN layers allow each pixel to gather information from farther regions.

1 layer → local smoothing (like 3×3 convolution)

2+ layers → broader context (like dilated convolution)

5. Use Cases

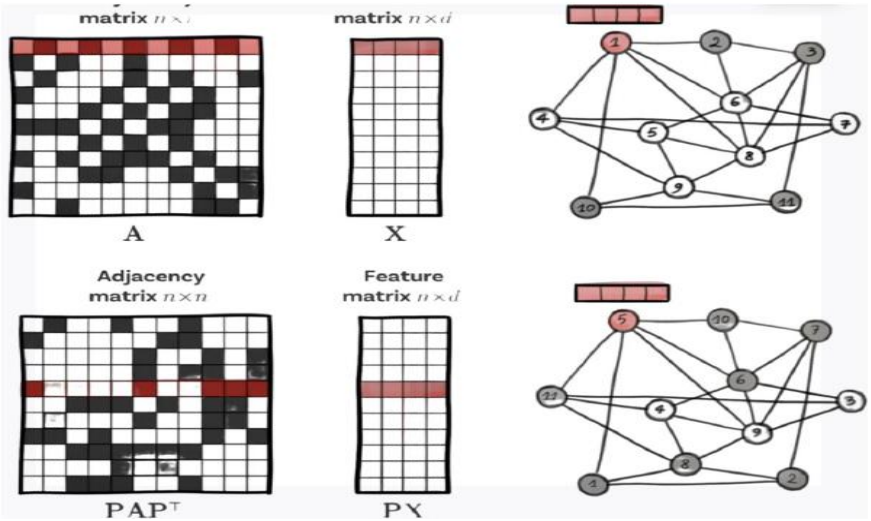
Image segmentation: Classify each pixel/node.

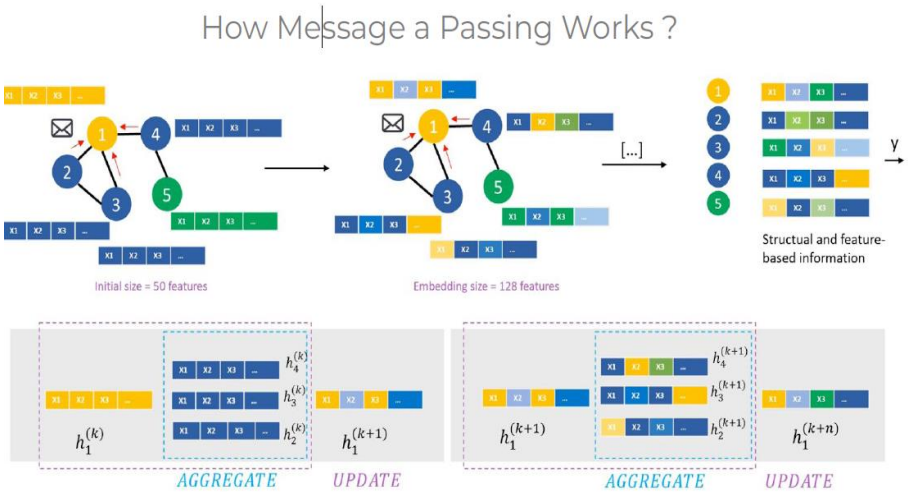
Super-resolution: Refine pixel values using graph context.

Denoising: Smooth noisy pixels using neighbor consensus.

Comparison: CNN vs GNN on Image

Aspect	CNN	GNN
Neighborhood	Fixed grid (e.g., 3×3)	Flexible graph (4 or 8 neighbors)
Structure	Regular	Irregular or structured
Kernel	Shared weights	Learned aggregation + update
Use case	Natural images	Images, meshes, irregular data

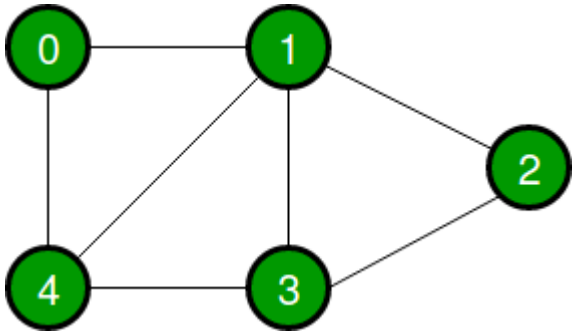




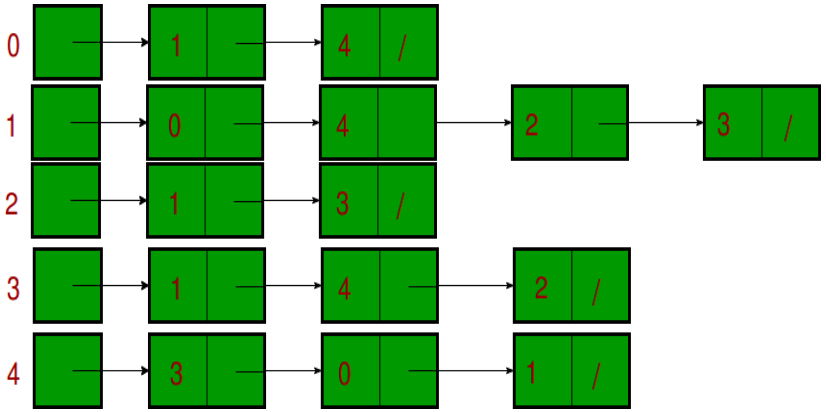
Example: in Facebook, each person represented with vertex(or node). Each node contains information like person id, name, gender, and locale. The an undirected graph with 5 vertices.

1.Adjacency Matrix is 2D array of size V ∗ V where V is the number of vertices in graph. To represent weighted graphs by a slot adj[i][j] = 1 indicates there is an edge from vertex i to vertex j.

2. Adjacency List used to represent a weighted graph, there is an edge from vertex u to vertex v are not efficient and can be done O(V).



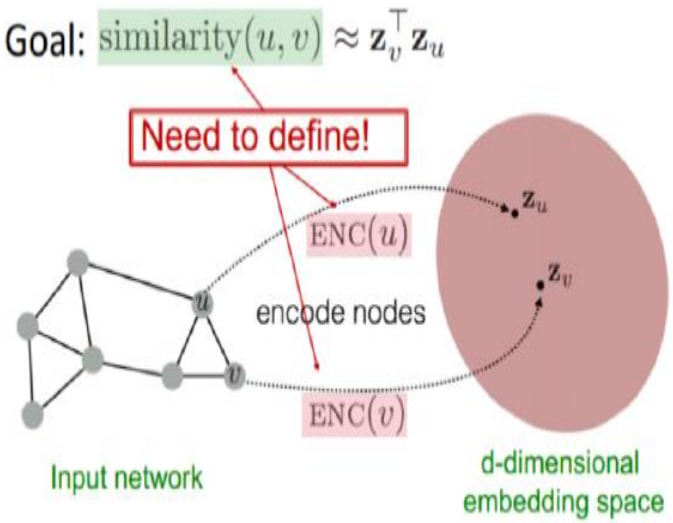
	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

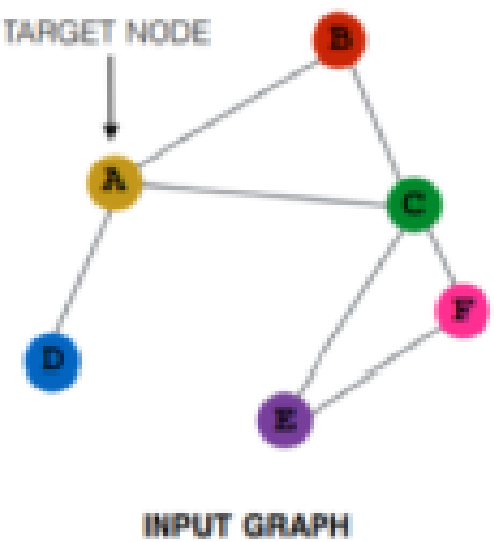


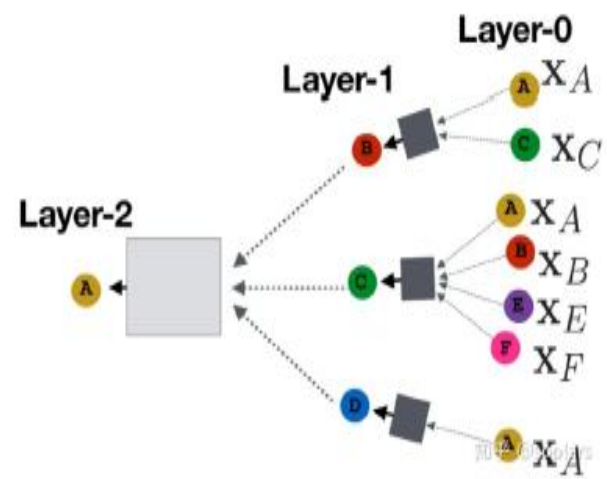
PageRank Algorithm:

algorithm created by Google to rank web pages, It calculates the importance of the page and assigns weights to each inbound based on two basic assumptions:

- Quantity assumption: if node receives more incoming links from other web pages, the page is more important.
- Let's define u and v as two nodes in a graph, x_u and x_v are two feature vectors. define the encoder function Enc(u) and Enc(v), which convert the feature vectors to z_u and z_v.







GNNs update node features recursively by aggregating information from their neighboring nodes through a series of message passing steps,

The message passed from node i to node j be defined as:

$m_{ij} = M(h_i, h_j, e_{ij})$, where h_i and h_j feature of node I, node j, and e_{ij} is the edge between them, M is a learnable function that computes the message to be sent

Once messages passed, they aggregated by each node to update its own feature representation by function as:

$$a_i = AGGREGATE(m_{ij}, \forall_j \in N(i))$$

the updated feature representation of node I

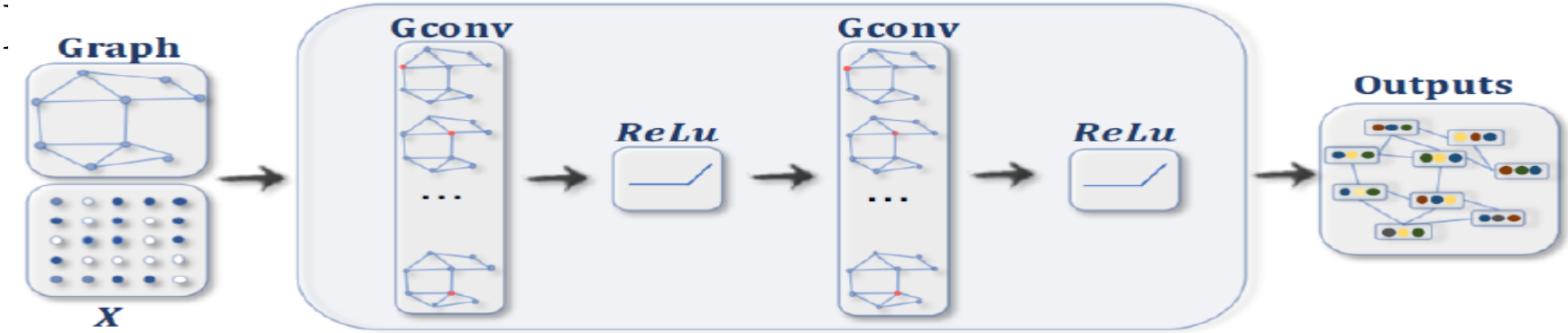
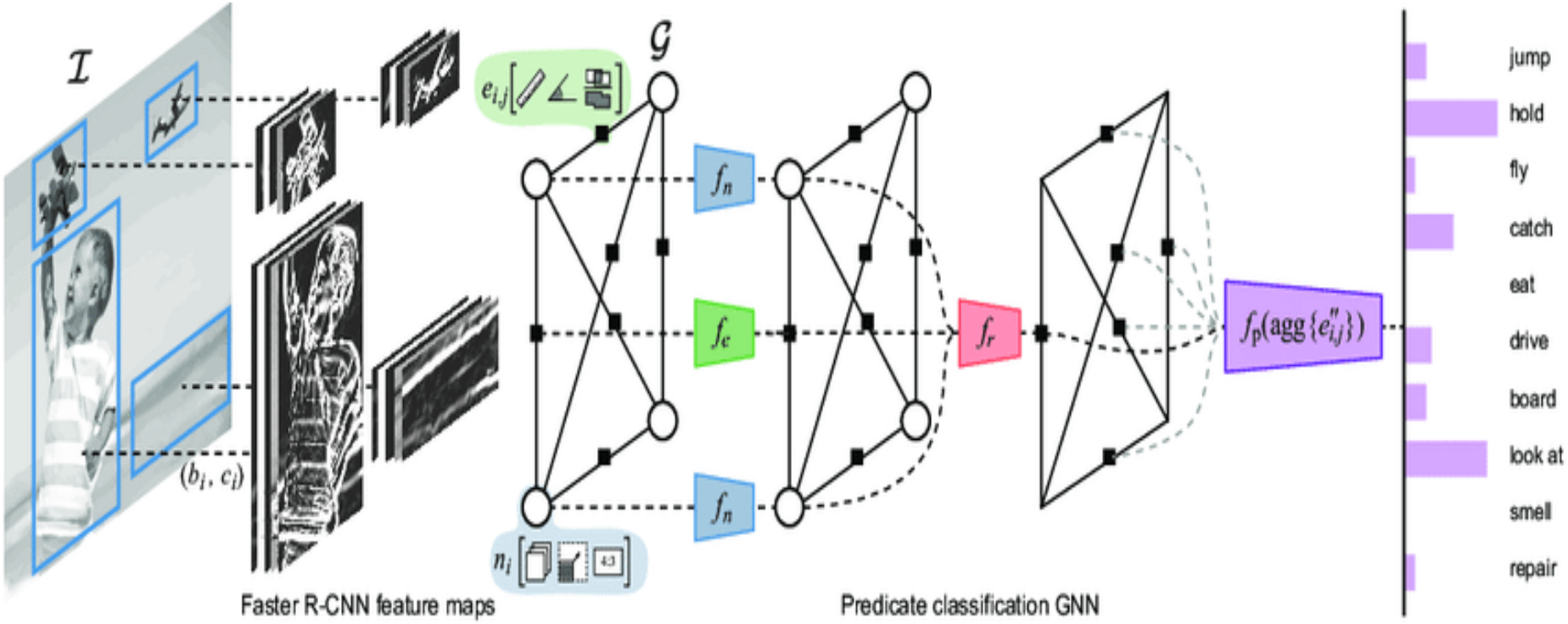
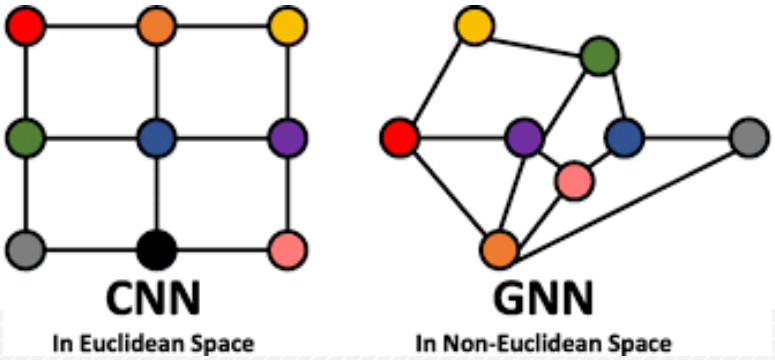
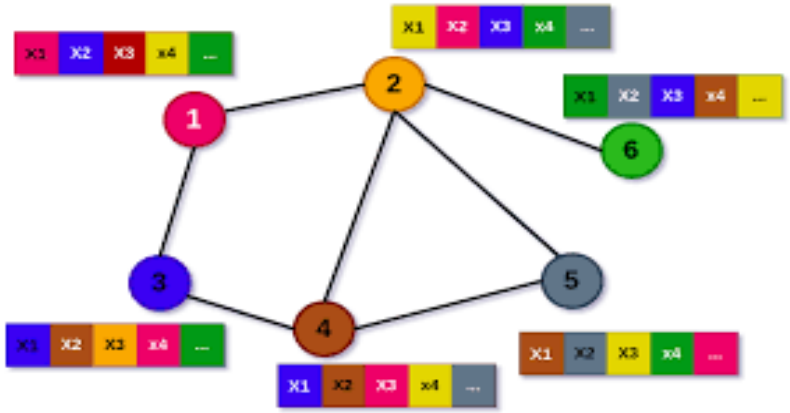


Figure 10: ConvGNN for node classification

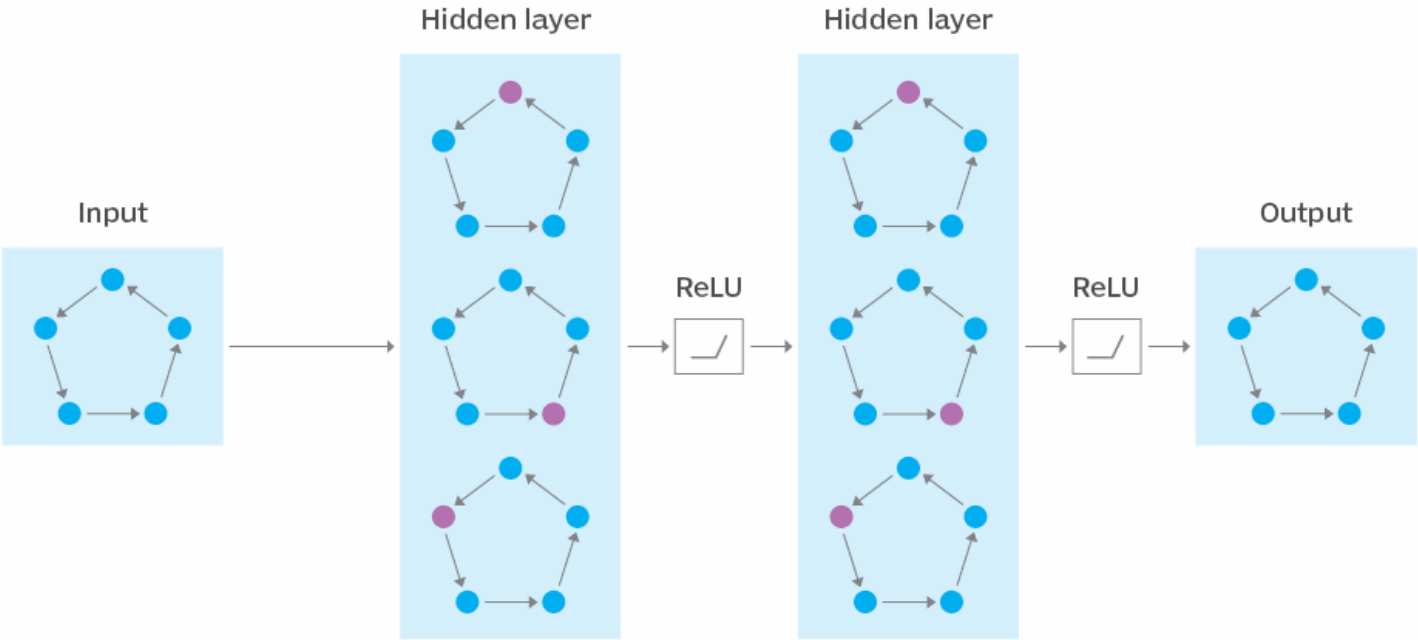


- The input representation and architecture implicitly encode an inductive bias for pairwise relationships.
- GNN trained to classify the predicates depicted in a scene. Object detections extracted through fully-connected graph Faster CNN.
- The GNN classifier aggregates local information across nodes and produces an image-level predicate prediction.

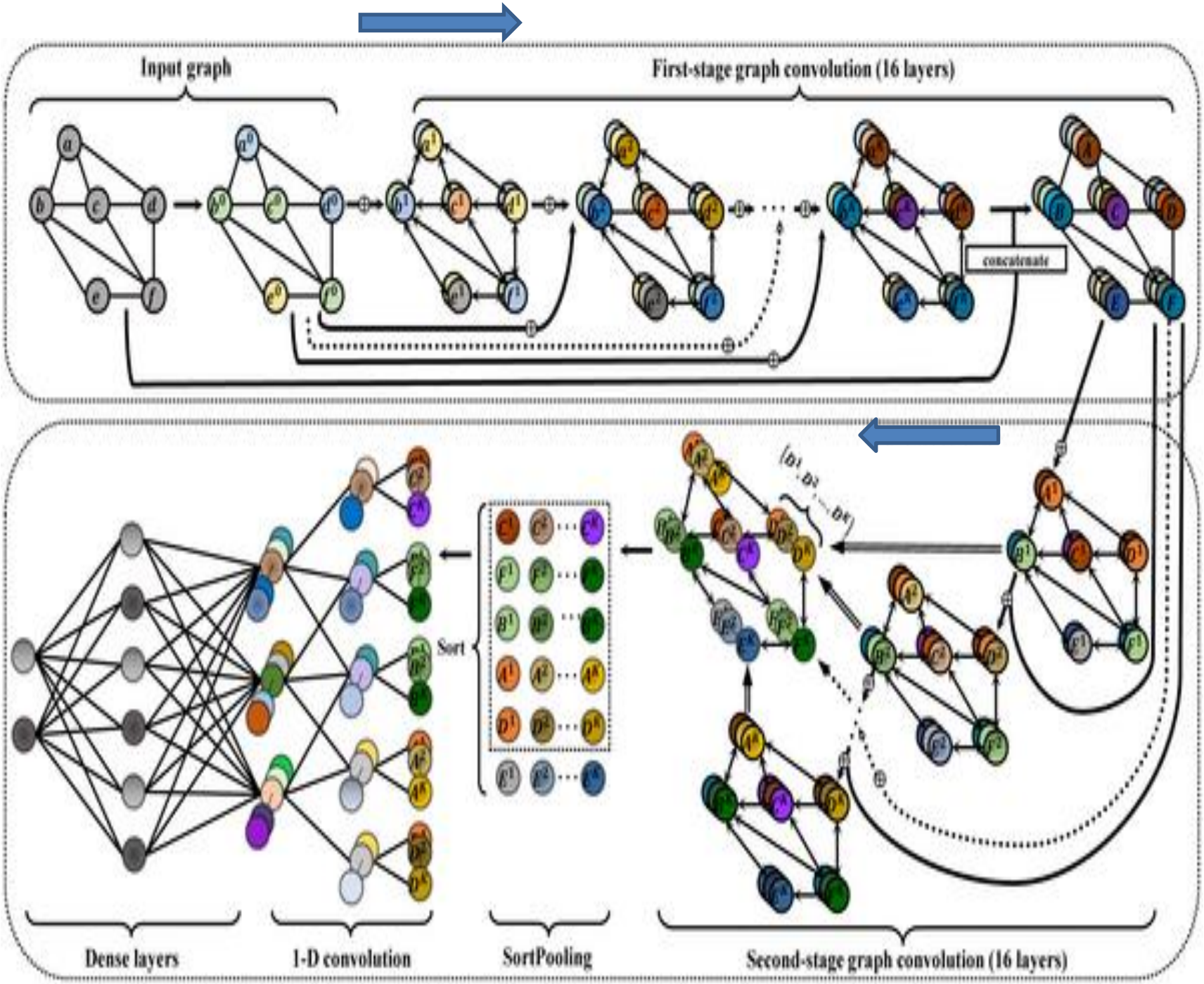




The structure of a graph neural network



©2024 TECHTARGET. ALL RIGHTS RESERVED TechTarget


Input graph
First-stage graph convolution (16 layers)
concatenate
Sort
Dense layers
1-D convolution
SortPooling
Second-stage graph convolution (16 layers)

What GMINN: Graph Matching Image Neural Network

designed to compare and match images by representing them as graphs. Each image is broken down into nodes (features, regions, or keypoints) and edges (relationships between those features). Then by correspondence.

How It Works

Step 1: Feature Extraction feature extractors generate descriptors for image regions.

Step 2: Graph Construction represented as nodes, with edges encoding spatial or semantic relationships.

Step 3: Graph Matching A matching algorithm (often based on message passing or attention) aligns nodes between two graphs.

Step 4: Neural Network Decision The network outputs similarity scores or classification results (e.g., “same object” vs. “different object”).

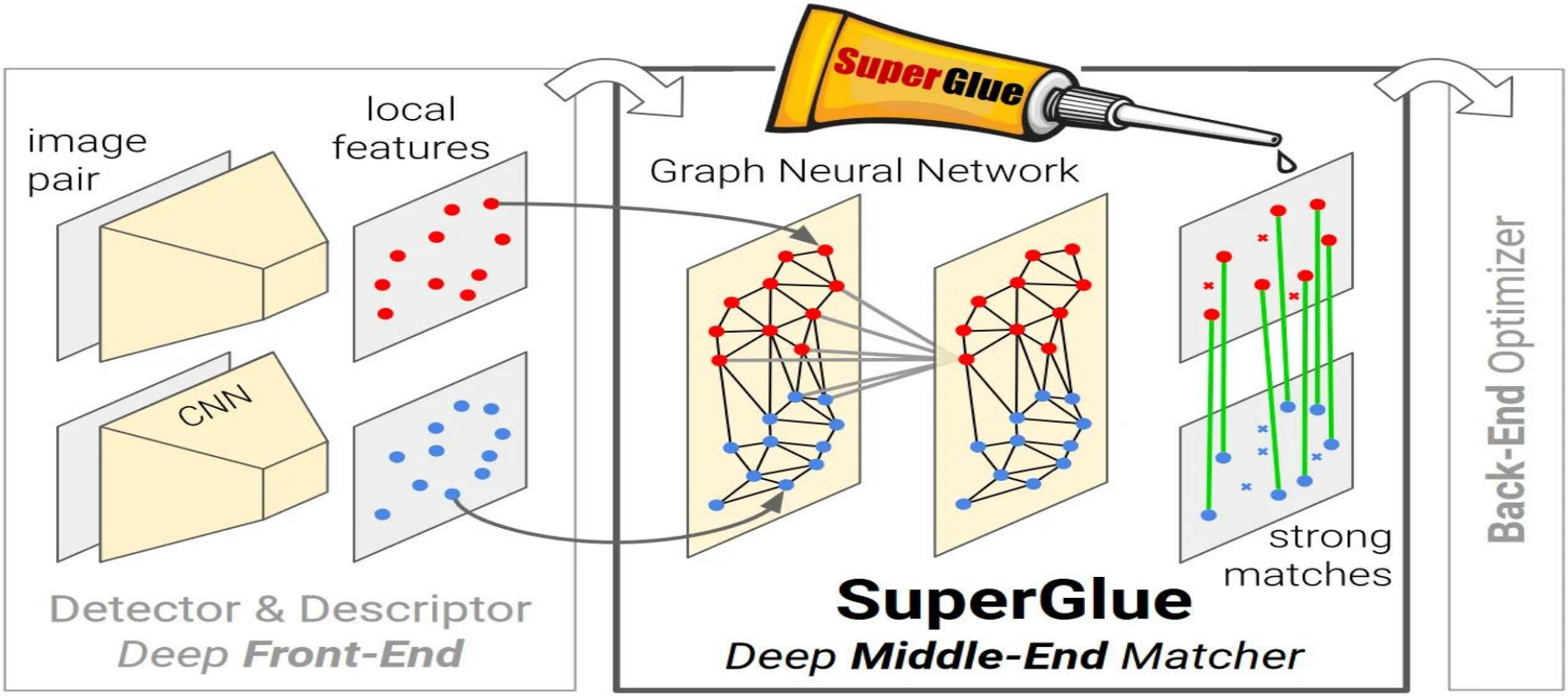
Applications of GMINN

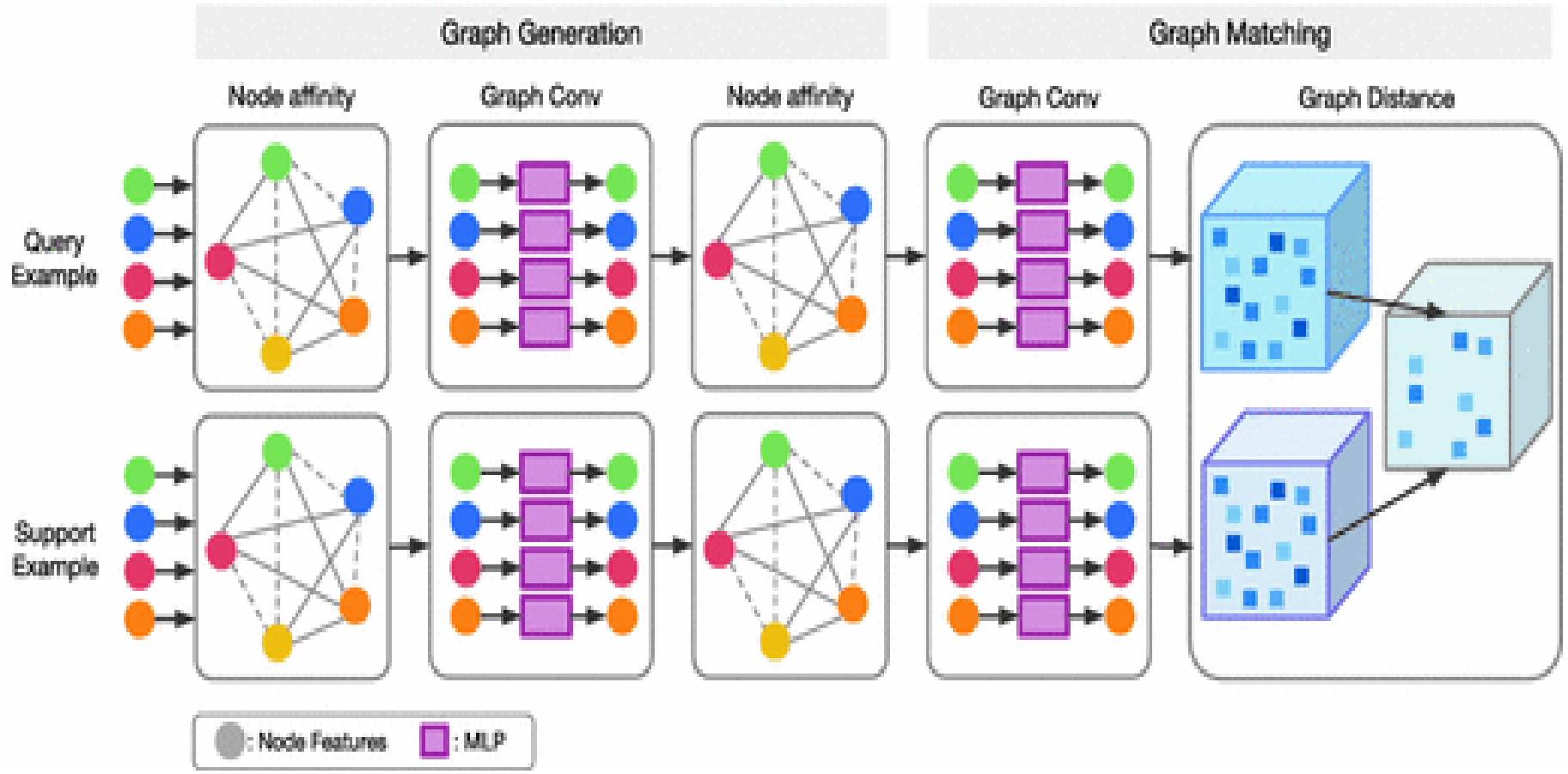
Image Retrieval: Finding similar images in large datasets.

Object Recognition: Matching objects across different viewpoints or conditions.

Medical Imaging: Comparing scans for diagnosis.

Biometrics: Face or fingerprint matching.




Graph Generation
Graph Matching
Node affinity
Graph Conv
Node affinity
Graph Conv
Graph Distance
Query Example
Support Example
: Node Features
: MLP

SuperGlue
Keypoints: 239:244
Matches: 210
Keypoint Threshold: 0.0050
Match Threshold: 0.20
Image Pair: 000000:000001